Short Tests Pick the Wrong Agent

ARC-AGI-3: a self-check ban led short tests, lost full runs; halving tokens for depth cost no score

Companion paper to the ARC Prize 2026 Paper Track writeup. Rusheel Sharma (Kaggle: rusheelsharma). Final submission (pick 1): `{{PICK1_VERSION}}`, public score `{{PICK1_SCORE}}` (mean of n draws). Public notebook: https://www.kaggle.com/code/rusheelsharma/arc-agi-3-short-tests-pick-the-wrong-agent-e154, version 1 (version 189's code cells byte for byte, plus a licence header cell), submission 57877052. Code and ledger: https://github.com/Rusheel86/arc-agi3-short-tests.

Abstract

ARC-AGI-3 asks an agent to learn unseen interactive games in which every environment action is scored against an efficient human, while thinking is free. On Kaggle a small open-weights model plays 110 hidden games on one GPU in nine hours, offline, so thinking is bounded per game. We report three measurements and the argument that connects them. First, short tests pick the wrong agent: banning the words that open self-checks led within each game's first 40k tokens in four comparisons (two inside noise), then lost 20 levels at 60 minutes per game (floor 6) and about 10 points on the hidden games (E055, E058) [measured]. The argument is about horizons: thinking's cost per turn is immediate and its payoff arrives on later, harder levels, so a test shorter than deployment is biased toward agents that think less, the short-horizon bias known from meta-optimisation (Wu et al., 2018). We observed this reversal; we did not predict it. A registered half-clock draw on the hidden games (E149 v4) then fell inside its predicted band (15.06; elasticity 0.93 +- 0.11), so tokens per game keep paying there. A registered test of the principle itself (E157) is void by its own mechanism gate: the post-trained model it tested kept 0.93x our reasoning tokens (Section 4.4). Second, a halving of tokens per game did not cost score: our final stack (E098) deepened each turn, which as an unplanned side effect halved its tokens per game (its registration had assumed x0.9) [computed], and it scored 28.11 over six draws against 23.98 over one (E098, E081) [measured]; five changes moved together, so what bought the +4.1 is not isolated. The two measurements agree: the ban cut thinking per turn and lost, while E098 deepened each turn at fewer tokens per game and did not lose. What bounds tokens per game is the KV-cache pool (games at once times context) and, for depth, attention per token at long context (x0.89 predicted from fewer streams, x0.49 measured) [inference]. Third, a single leaderboard draw is uncertain by about +-3 points at 90%, and a true +3 needs about six draws per version (E138). We also report a keep-alive version for slow platform loads (E154: four draws, mean 28.89) and every pre-registered idea that failed, with its kill number.

1. Introduction

Most published ARC-AGI-3 systems use frontier API models on the public games. The Kaggle competition removes both: one RTX PRO 6000, nine hours, no internet, 110 unseen games. A team chooses an open model once and then spends its effort on how the model is served, how its thinking is spent and how candidate changes are tested; this paper is about the last two. We started from Daniel Franzen's open Milestone-2 notebook (27.80-27.89; E001) [measured], built on Tufa Labs' Duck harness. Our final stack scores about the same (Section 7), so we claim no score gain over it. We contribute measurements that should change how others test changes to an agent's thinking and how they read a leaderboard:

- **C1, the horizon argument and its record (Theory).** A test shorter than deployment counts thinking's cost and misses its payoff, so it is biased toward agents that think less (Section 4). The argument was stated after the observation it explains; its record, including two registered tests of the condition it needs, is in Section 4.4.

- **C2, the horizon reversal (Novelty).** A change that cuts thinking leads on short tests and loses at full length, on public and hidden games; short screens select the wrong agent (Section 5).
- **C3, what bounds tokens per game (Theory, Progress).** The KV pool bounds games at once times context, long context raises the price of each token, and a halving of tokens per game, a side effect of deeper turns, did not cost score (Sections 4.2, 4.4, 8).
- **C4, the noise floor (Accuracy, Progress).** Repeated draws of one version on both ARC Prize 2026 boards and the decision rules they support (Section 7).
- **C5, the engineering (Completeness).** State hand-off, a 4-bit-value KV cache spent on depth, a timeout fix and a keep-alive for slow loads, each with its kill rule (Section 6).
- **C6, the negatives (Progress).** Every pre-registered idea that failed, with the number that killed it (Section 10).

2. Setting and scoring

Games and score. Each game is a small grid world whose rules the agent discovers by acting; every game has at least six levels, level 1 a tutorial [read: competition documentation]. A level scores $\min(1, (h/a)^2)$, with h an efficient human's action count and a the agent's; unfinished levels score 0. A game's score weights its levels by level number, and the total is the mean over games. Only environment actions count, RESET and UNDO included; thinking and code are free; Kaggle has no five-times-human cutoff [read: discussion 713921]. Points are lost mainly on levels never reached, not on extra moves [measured on the baseline, E009].

The box. One RTX PRO 6000 for nine hours, no internet, 110 hidden games (55 on the public leaderboard, 55 private), open-weights models only [read]. Each game can be opened once and resets are level resets. The harness releases the games about 12 minutes into the notebook and gives each 532 minutes (E064) [read in code]; in our stack 19 games think at once and the rest wait for a slot (E098) [read].

Draws. One submission per day is allowed and two count for the final private board [read: ledger header]. Each submission reruns a saved version from scratch on the 55 public-leaderboard games, so repeated submissions of one version are independent draws of the same agent; we never pool draws of different versions. Everything that does not need the model was tested on a laptop with the official toolkit and a stand-in model server (`localrig/`).

3. Inherited stack and credits

The agent is Tufa Labs' Duck harness as extended by Daniel Franzen, served by Lord Han Solo's vLLM build. Table 1 lists every inherited component; our additions are in Section 6.

Table 1. Inherited components (ARC-AGI-3 notebook, then the ARC-AGI-2 notebook).

Component	Author	Role in our notebook	Licence
Duck harness	Jeroen Cottaar, Tufa Labs	the core loop: the model plays through a Python tool, inspecting the board, testing mechanics and acting from code	Apache-2.0 [recorded]
Milestone-2 harness, prompts, scheduler	Daniel Franzen	animation frames, 10x images and difference images, UNDO, no-op guards, retained functions across calls, larger context, a scheduler that gives more work to games that progress	Apache-2.0 [read: vendor/franzen-m2/LICENSE]
vLLM runtime (commit e975732 + overlay) and the mixed NVFP4/FP8 checkpoint of Qwen 3.8 Flash Next	Lord Han Solo	the model server: MTP speculative decoding, QSA attention, offloaded PLE table	Apache-2.0 [recorded]

Component	Author	Role in our notebook	Licence
4-bit KV-cache patch	andreasgru	basis of our K8V4 port (Section 6.3)	Apache-2.0 [read]
Qwen 3.8 Flash Next weights	Qwen team	the model (125B total parameters, mixture of experts)	Qwen Community License 1.0; prize eligibility open [recorded]
failed-in-aimo notebook (ARC-AGI-2 v4 = a byte-identical copy)	koushikrudra	4B grid model, per-task test-time training (TTT), DFS decoding, vote selection	Apache-2.0 as recorded 10-21; page to be re-checked
qwen3_4b_grids15_sft139 (NVARC 4B)	Ivan Sorokin, Jean-François Puget (NVARC)	the ARC-AGI-2 grid model	Apache-2.0 [recorded]; NVARC code and data state no licence and are not redistributed
ARChitects 2024 code	Daniel Franzen, Jan Disselhoff	lineage of the AGI-2 notebook's loader, decoder and solver [inference, not diffed]	Apache-2.0 + NOTICE [verified 10-25]
icecuber ARC-solution; arc-dsl	top-quarks; Michael Hodel	private measurements only (E141, E144), not submitted	MIT [verified 10-25]
BARC seeds	Li et al.	private measurement only (E144)	no licence: results reported, files not redistributed

We trained no submitted weights [read]. Our own code is released under MIT-0, as arcprize.org requires [read 10-25]; inherited files keep their licences and notices.

4. Why short tests pick the wrong agent

4.1 The argument

Moves are scored, thinking is free, and compute is fixed. A level scores only when finished, and later levels combine earlier mechanics [read: competition documentation]. Thinking's cost is immediate: each turn's tokens come out of the game's budget, so an agent that thinks less finishes easy levels sooner. Its payoff arrives later: the checks that look wasted on an easy level are what finish a hard one. A test shorter than deployment counts the cost and misses the payoff, so it is biased toward agents that think less.

The same structure has a name in meta-optimisation. Wu, Ren, Liao and Grosse (ICLR 2018, arXiv 1803.02021) show that tuning learning rates by gradient over short unrolls of training introduces a bias toward small step sizes, which they call short-horizon bias: even with 100-step horizons, the chosen rates were orders of magnitude too small [read: abstract, 10-30]. In our setting the screen is the short unroll and the nine-hour scoring run is the full training run; "think less" plays the part of the small step. The analogy is ours [inference]; their analysis of why the bias arises is on a noisy quadratic model and is not used here.

Both halves are visible in the ban's 60-minute runs (E058; 18 public games, one session; Section 5.5). On levels both arms solved, the banned agent needed 11% fewer tokens: the cost of checking was real. Past 45k tokens on one level, where the registered rule looked (E058 rule A: finishing at under half the control's rate with at least 10 levels at risk in each arm), the banned agent finished 2 of the 22 levels still open in the 45-60k band against 8 of 21 (0.64 vs 3.38 levels per 100k tokens): so was the payoff [measured]. Later bands are closer (60-90k: 1.01 vs 1.21; past 90k: 0.66 vs 1.63) [measured]. At 20 minutes per game the ban gained 9 levels (61 vs 52; E055) [measured], though E055's control was a weak draw: against two other unbanned runs of the same setup its gain in total levels was +4 and +2, inside the day-to-day spread [measured, E055 revision; E058]. At 60 minutes it lost 20.

The argument says which way a short test errs, not that more thinking always pays or that every cut leads early. Lowering reasoning effort to medium (E042) lost outright at 20 minutes, with no early lead (Section 5.6): a thinking cut that a short test would already have rejected, and a counter-case to reading the reversal as general (Section 13).

4.2 What bounds tokens per game

The argument needs a budget, and on one GPU the budget is set by memory and by the price of each token. A decode step reads the weights once for the whole batch plus each stream's KV cache [read: standard serving arithmetic], so throughput rises with the streams decoded together until the cache fills (E040: about 21.7 ms + 1.06 ms per stream at 7-14 streams [measured]; above 24 streams at long contexts the per-token cost grows instead, E084 [measured]). The KV cache is cut into fixed-size pool blocks (PagedAttention; Kwon et al., 2023), so the pool decides how many games think at once and how much context each keeps. Two costs must be kept apart: the pool bounds games times context, while a longer context also makes each generated token slower. When our final stack bought depth (a longer window and longer replies), its generation rate fell to x0.49 of its predecessor's; fewer streams alone predict x0.89 (E040's step model, fitted at short contexts), so most of depth's price was attention per token at long context [inference; Section 8.1]. By "memory" we mean the KV pool that bounds games times context, and we report the per-token cost as the price of depth. Every serving lever has a memory price as well as a speed price (Section 8.6).

4.3 Public-game returns to tokens

We truncated two 60-minute runs of 18 public games (the controls of E058 and E061: no ban, 18 at once, 92k window) at fixed token budgets per game and scored each truncation with the competition rule (E072, T3 recheck) [measured].

Table 2. Pooled mean score on 18 public games within B generated tokens per game (two 60-minute controls).

B (tokens per game)	20k	45k	80k	100k	140k	180k	195k
Score	3.12	12.29	21.96	28.48	36.83	42.75	44.33
Elasticity of the step				1.16	0.76	0.59	0.45 (0.75 / 0.12 per run)

Between 140k and 180k the elasticity is 0.59 (game-cluster bootstrap 90% 0.33-0.89): +10% tokens, about +6% score [measured]. Returns fall with the budget; the last 15k step (180k to 195k, a handful of levels) is unstable between the two runs (0.75 vs 0.12) and bears on the saturation reading of Section 8.2 [measured, `analysis/t3_recheck_results.txt`]. The 60-minute noise floor is about 6 levels or 8 points (two identical-setup controls: 82 vs 77 levels) [measured, E072]. A caveat on the method: truncating one run at B is not a run that was given B. The model reads its game clock and paces itself by it, which is why the hidden-set dose test (E149) scaled the clock the model sees instead of cutting the run [read, E149].

4.4 What the argument implies, and its record

- **P1 (horizon).** A change that cuts thinking per decision looks good while the budget is short (easy levels finish sooner) and can lose once it is long enough for hard levels. E058's registered prediction gave this loss 25% (no effect 55%); the reversal was observed, not predicted, and P1 is stated after it [read, E058].
- **P2 (the token leg).** The argument needs tokens per game to keep paying at deployment length: if returns had stopped there, cutting thinking would cost nothing at length. Two tests registered before their data check it, E149 v4 on the hidden games and E155 on community games at deployed depth (Section 8.5). E149 v4 passed: 15.06, inside its band, elasticity 0.93 +- 0.11 [measured, computed]. Both test this premise, not the principle itself; P5 does.
- **P3 (depth).** Depth (a longer window, longer replies) costs tokens per game; at elasticity 0.59 a halving costs about a third of the score (0.5^0.59 = 0.66) [computed] unless depth pays. A deeper version scoring no

better than the shallower one would have contradicted the bet (E091's registered CONTRADICTED reading; Section 8.4) [read].

- **P1 and P3 together.** The ban cut thinking per turn and lost at length. E098 deepened each turn (32k-token replies, a longer window), which halved its tokens per game as a side effect (its registration assumed x0.9 tokens per game; Section 8.4), and did not lose. At elasticity 0.59 halving alone predicts about 16 points against E081's 23.98 (Section 8.2), so depth, with the four changes that moved with it, paid at least that predicted loss [inference; not isolated]. The two measurements read the same way: cutting thinking per turn cost score at length, while fewer tokens per game that came with deeper turns did not [inference].
- **P4 (speed levers).** A speed lever is worth the memory it frees or costs, not its single-stream speed (Section 8.6).
- **P5 (the principle, prospectively; E157).** Registered on 2026-10-30 before any output existed: ThinkingCap, our served model after BottleCap AI's "think less" post-training (their claim: thinking -22.5% on tau2-bench at 83.39 vs 83.57, -38.5% at macro accuracy 88.8 vs 89.8), grafted into version 189's stack with everything else unchanged [read, E157]. The principle predicts that a model thinking less per turn loses at full length, so the registered prediction is a draw y below version 189's mean 28.89. Bands, from the skeptic's review (one draw against the n-4 mean, SE of the difference 2.05): CONFIRMED (a collapse) if y < 24.0 (P 0.01 under no effect, 0.94 under a true -8); NOT CONFIRMED if y >= 27.0 (P 0.82 under no effect); INCONCLUSIVE between. VOID, checked first: an ERROR status or y < 5, a failed smoke, or the mechanism gate M (mean reasoning tokens per response in the smoke at or above 0.9 x E154's 942.2, i.e. the post-training did not cut thinking at our template). One draw was registered as underpowered for effects under about 3 points, with no gain claim [read, E157; reports/skeptic_e157_2026-10-30.md, Decisions]; at 90% power, one draw against version 189's four detects about 6.0 points (one-sided 0.05, SD 1.83) [computed], so it was underpowered below about 6 and could detect only a collapse. Its kill check passed on the laptop: only the attention projections and the shared experts changed; the routers (read whole) and the MTP head (sampled) are byte-equal, so the base model's prune plan still applies; the served FP8 and NVFP4 recipes were reproduced byte-exact [measured, E157 K laptop part]; the graft kernel's full census confirmed that the routed experts and the MTP head are unchanged [measured, E157 graft kernel v3]. Named confounds: requantising a fine-tune, and one draw tests only the full-length half of the argument, not the early lead (the skeptic's E157b design, a paired community-game run, would show both; not run). Result: VOID [measured, E157 'Result, G2 smoke', 2026-11-01]. The smoke (kernel version 192) passed every technical gate: the pinned graft manifest, pruning and K8V4 on at 19 sequences, 348 generated tokens per second against the 281.5 floor, MTP acceptance length 2.54. Its mean reasoning tokens per response were 872.7 (median 615, 110 responses) against the registered reference 942.2: a ratio of 0.93, at or above 0.9, so by the registered rule the theory test is void. At our template the post-training cut thinking by only about 7%, and the base model's own two smokes read 942.2 and 1,179.2, so smoke-to-smoke spread covers the gap [descriptive]. The principle remains untested prospectively. As registered, the version stays only a pick-2 candidate; its draw is read descriptively, never pooled, with no theory claim either way: `{{E157_DRAW}}`

Record: E149 v4 has passed (Section 8.5); E155 is pending; E157, the only registered test of the principle itself, is void (P5). E091 was never run as registered, E098's registered 22-27 was exceeded, and E149's first version is void [read: E091, E098, E149].

5. The horizon reversal

5.1 The change

Without intervention, most sentences that open with "Hmm", "Wait" or "Actually" in our agent's thinking are checks of a stated rule against cells [read, E058 transcript analysis]. E055 banned four words ("Hmm", "Wait", "Actually", "Alternatively") at the sampler (vLLM bad_words; logit bias is refused under speculative decoding)

[recorded]. The ban removed them (0.00 per 1,000 kept characters) without substitutes appearing, and cut tokens per model call from 1,550 to 1,172 (x0.76) [measured, E055].

5.2 Early lead

On the same 18 public games the ban finished more levels within each game's first 40k tokens in all four comparisons [measured]:

- against the same-session control of a 20-minute screen (E055): +19 (48 vs 29; 10 games ahead, 3 behind);
- against two earlier 20-minute unbanned runs of the same setup (E040, E051): +8 and +14 (sign-test p 0.75 and 0.04);
- inside the 60-minute run of E058: +4 (10 games ahead, 2 behind).

One-copy differences of levels within 40k have an SD of about 8.5 (E058 noise note) [computed], so +8 and +4 are inside noise. Only E055 was a registered test, and its +19 was inflated by a weak control draw; total levels after 20 minutes were +2 to +4 against typical unbanned draws, inside the day-to-day spread [measured, E058]. The ban was promoted on its mechanism and early lead, and submitted.

5.3 Loss at full length

E058 then ran both arms for 60 minutes per game on the same 18 games in one session (about 190k tokens per game). The ban lost: 62 levels against 82 (0 games ahead, 11 behind, 7 equal; sign test $p < 0.01$), mean score per game 29.31 against 51.57 by the harness's own scoring (paired -22.26, 90% bootstrap -33.8 to -12.5), games won outright 1 against 6 [measured]. Figure 1 scores the same runs with the simulator rule of `analysis/t3_recheck.py` (also used for Table 2), which ends at 27.59 against 49.78 [measured, fig2 data]. The lead turned around between 40k and 60k tokens per game: +4 within 40k, -4 within 60k and 100k, -10 within 150k, -20 at the end [measured]. The 60-minute floor is 6 levels (E072) [measured]; two 20-minute copies inside the E058 session differed by 2-3 levels [measured]. Figure 1 shows the curves.

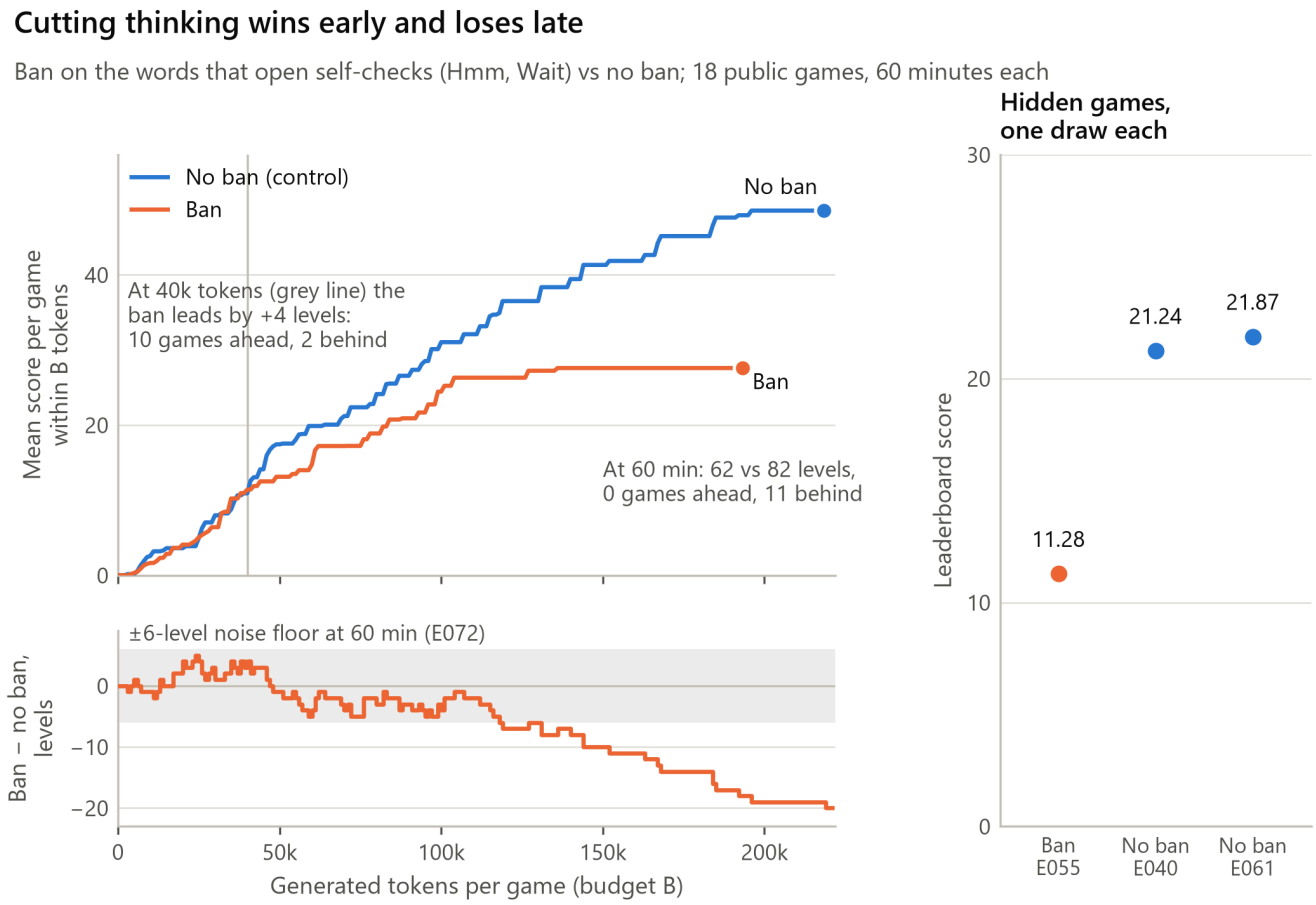


Figure 1 (the cover, cover_v6.png, is this figure reduced). Short tests pick the wrong agent. Self-check word ban vs control, 18 public games x 60 minutes (E058) [measured]. Score is by the simulator rule of the truncation analysis (`analysis/t3_recheck.py`, E072), which ends at 27.59 vs 49.78; the harness rule used in the text ends at 29.31 vs 51.57. Middle: level gap within B tokens per game; floor +-6 levels (E072). Right: hidden-set draws, one each, not identical code (E055, E040, E061) [measured].

5.4 On the hidden games

The ban version (script 356099442) drew 11.28, against 21.24 and 21.87 for the nearest unbanned vLLM versions (scripts 355734881 and 357018543, E040 and the E060/E061 notebook) [measured]. These are one draw each and not identical code: the ban version and the 21.87 version carried the draft vocabulary and the 21.24 version did not, and an earlier ledger entry had labelled the 21.24 draw as the E044 effort bandit before the 10-09 correction assigned it to E040 [read]. Because the versions differ, the relevant spread is between versions: the old vLLM versions (one draw each) spread with SD 2.43 (E138) [computed], so a gap of about 10 points is about 2.9 standard deviations of a two-draw difference [computed]. The direction holds; its exact size does not.

5.5 Mechanism

With the ban the agent wrote 36% less thinking per step (4,242 vs 6,667 characters), took more steps and moves, and generated 12% fewer tokens per hour [measured, E058]. On levels both arms solved it needed 11% fewer tokens, so the speed on easy levels is real; the loss is on hard levels: in the registered band past 45k tokens on a level (45-60k), 2 of 22 levels at risk finished against 8 of 21 (0.64 vs 3.38 per 100k tokens; later bands 1.01 vs 1.21 at 60-90k and 0.66 vs 1.63 past 90k) [measured]. Revision phrases the ban does not touch ("let me reconsider") also fell, from 0.42 to 0.27 per 1,000 characters: the checking did not move elsewhere [measured]. On cd82 level 3 the unbanned run used 15 moves and 192 rule-check sentences and solved it; the banned run used 130 moves and 28 and did not [measured; an illustration].

5.6 Effort, and what had been predicted

Lowering reasoning effort from xhigh to medium (E042, 25 public games, 20 minutes) cut tokens per call to x0.88 and lost outright, 45 levels against 53, with no early lead (within 40k: 0) [measured]; within games medium gave x0.74 completions per token (90% 0.53-1.04; E044) [measured]. Both "think less" changes lose at length; only the ban showed the reversal.

The registered prediction gave a loss of 12 or more levels 25% ("no effect within noise" 55%, "ban ahead by 12 or more" 20%) [guess, written before the run, E058]; the outcome, -20, surprised us.

5.7 Prior art and what is new

On single-answer mathematics the loss in small models is known. NoWait (arXiv 2506.08343) bans reflection words with little loss on 32B models trained with RL (QwQ-32B AIME24 73.33 to 71.33 at 16% fewer tokens; other QwQ cells rose), but distilled Qwen3-4B, 8B and 14B lose 13.33, 14.61 and 16.67 AIME25 points (its Table 6) [read: PDF, 10-25]. CGRS (arXiv 2508.05337) suppresses only when confident, cutting tokens 18.5-41.9% for up to about 3 points; its blanket-suppression row shows the cost of banning outright (R1-Distill-Qwen-7B, AMC23 87.5 to 76.7) [read]. Forcing "Wait" lifts s1's AIME24 from 50 to 57 (arXiv 2501.19393) [read]; token-level suppression hurt a 1.5B model (arXiv 2605.28305) [read: HTML summary only]; ThinkBrake (arXiv 2510.00546) stops thinking early in single-call tool use [read].

That a model's standing depends on the token budget is also known for single answers. De Souza and Panisson (arXiv 2608.12150) cap generation at seven budgets from 64 to 4,096 tokens for four models (LLaMA-3 8B, Qwen-3 32B, LLaMA-3.3 70B, GPT-OSS 20B) on GSM8K, MATH-500 and GPQA-Diamond; model rankings reverse across budgets on all three benchmarks, and 3-19% of items get worse with a larger budget [read: abstract and HTML, 10-30]. Each item there is one answer scored at a cap: a cut cannot compound across levels, and no action is scored.

In agents, less reasoning helped on SWE-bench, where actions are not scored (Cuadron et al., arXiv 2502.08235) [recorded: secondary sources]. In web agents, scaling interaction steps beat deeper per-step reasoning (TTI, Shen et al., arXiv 2506.07976): with Gemma 3 12B prompted on WebArena, more steps raised success from about 23% to at least 28%, while per-step budget forcing plateaued near 26%; the reward is task success only, with no step penalty described [read: arXiv HTML v2 through a fetch summary, 10-28; figures not re-checked in the tables]. There actions are not scored, and our ban, which took more steps and moves, lost at length. On ARC-AGI-3 itself, Liew (arXiv 2605.25931) names a speed-depth trade-off, but between action efficiency and the information gained by exploring, for a three-phase explore-verify-plan agent on Qwen2.5-0.5B; thinking budgets are not its variable [read: abstract, 10-30]. Searches through 2026-10-30 (the evidence check of 10-25, the skeptic reviews of 10-27 and 10-30, and a web search of 10-30) found no test of reflection-word suppression in a long-horizon agent with scored actions.

What is new is therefore narrow: the reversal over a game's horizon in an agent whose moves are scored. The same change wins the short test and loses the long one, so a short screen selects the wrong agent. Budget-dependent rankings (de Souza and Panisson) and short-horizon bias (Wu et al.) are not new; we add the measurement in an agent whose cuts compound across levels, its mechanism (Section 5.5), and the cost of having trusted the short screen (a submitted version that drew 11.28). A reusable check: the post-hoc flag "thinking characters per step fell" caught every ban-family arm with 0 false alarms in 9 null pairs (E072, T2) [measured, post hoc]; a change that trips it should be judged at the deployment horizon. Caveats: one 60-minute session, one copy per arm, 18 public games, and hidden-set draws of one per version with different code.

6. Method: what we added

Each change below was registered in the ledger with a kill rule before its first run.

6.1 Expert pruning calibrated on the agent's own traffic (E060, E073)

We counted which experts the agent's own requests use (E060 census) and kept 384 of 512 per layer. At equal tokens per game on 18 public games the pruned server finished 76 levels against 71 (registered rule: ADOPT) [measured, E073]. Its speed premise failed in play (987.0 vs 995.5 tokens per second at 18 busy streams, x0.991) [measured]; its value is memory: the pruned pool holds 2,540,420 tokens, so 24 games fit where the unpruned pool holds 18 [computed fit, E073]. Its one leaderboard draw, at 24 games and before the hand-off fix, was 18.10 [measured].

6.2 A constant-cost state hand-off (E077, E081)

The harness rewrote the whole game history (every 64x64 frame as text and grid) after each move and reloaded it for every Python call: CPU time per step grew from about 2 s before move 100 to about 26 s after move 400, and the share of game slots generating fell from 0.97-0.98 early to 0.74-0.84 in minutes 40-57 [measured, E077]. The fix keeps an append-only history and an incremental sandbox state. Replays through the full harness gave byte-identical model inputs (0 of 41, 35 and 111 requests differ) with the 30-second tool cap lifted to 600 s in both arms; under the normal cap the old code timed out where the new did not, so the hand-off changes what the model sees only through fewer late timeouts [measured, E077-A]. On Kaggle, occupancy in minutes 40-57 rose from 0.741 to 0.980 [measured, E081]. E081 (pruned, 24 games, 92k window, 12k reply cap, hand-off) generated 1,112 tokens per second and drew 23.98 (one draw) [measured].

6.3 A KV cache with fp8 keys and 4-bit values, spent on depth (E083, E091, E098)

We ported andreasgru's 4-bit KV patch to Lord Han Solo's stack with fp8 keys and NVFP4 values ("K8V4"). On the agent's own traffic it adds +0.0016 nats per token (bar +0.004; 4-bit keys and values together: +0.008), passes 5 of 5 needle checks, grows the pool x1.18-1.24 and leaves decode speed unchanged (1,205.4 vs 1,202.2 tokens per second), with prefill 22% slower [measured, E083 run 2]: a memory lever, not a speed lever.

We spent that memory on depth. E091 raised the reply cap from 12,288 to 32,768 tokens and the window from 94,208 to 160,768 (81,408 kept after a trim instead of 40,704), which fits 15 games in the fp8 pool [read, E091]. With K8V4 it fits 19: each game needs 83 blocks of 2,128 tokens and the pool holds 1,636 [computed, E098]. Width was already closed: throughput fell above 24 streams (x0.927 at 32; E084) [measured].

6.4 A timeout fix (E097, E098)

A 30-second sandbox timeout wiped every helper function the model had written for that game, at 6 of 6 timeouts in E081 and 16 of 16 in E058 [measured]. E098 keeps them (one flag of the E097 patch; E097's fact flags stay off, as E097 showed no gain: 73 vs 76 levels) [measured]. This bug fix has no relation to memory and moved together with depth.

6.5 Surviving slow platform loads (E146, E154)

Stub commits and their failure. Saving a Kaggle version runs it once ("commit"); a full GPU commit of our notebook costs 30-35 minutes. E146 made the commit a 15-second check that writes a valid placeholder submission, while the competition rerun runs the full pipeline; a save cost 0.03 GPU-hours [measured]. The one stub-saved version ever scored (kernel version 187, ref 57606176) drew 0.00; its scored file had 3,411 bytes, like a 10-05 notebook that never played, against 3,773-4,100 for the 17 scored draws: the file of a run in which no game completed a level [measured]. Kaggle staff confirmed slow model reads in those days; other teams reported 0.00 runs of unchanged versions and a mechanism of 35-63-minute loads plus a gateway rule that

closes a scorecard after 15 idle minutes [read: forum 751598, 752123, 750173]. That rule is our suspected cause, not a determined one [guess]. The stub was never validated, so the final candidates were saved with real commits.

The keep-alive (E154). E098 releases the games at minute 12 with one warm-up RESET and then waits for the server, so a server not answering by about minute 27 would leave the scorecard idle for more than 15 minutes [read]. E154 adds three timers and changes nothing else on the competition path [read; tests 66/66]:

1. **Keep-alive.** The warm-up game polls the server's health URL every 10 seconds and, while it does not answer, sends one RESET every 7 minutes; it stops at the first healthy answer, after which play is E098's.
2. **Progress-aware start cap.** E098 abandoned any server start not ready at 45 minutes and restarted on a slower fallback that re-reads the 167.7 GiB checkpoint. E154 abandons a start only after 15 minutes without progress (log sizes and disk use sampled every 60 s; a normal start is never silent for more than 42 s [measured]) or at a hard cap of 100 minutes. The launcher's ready deadline rises from 3,600 s to 6,600 s (110 minutes), rerun only.
3. **First-request grace.** The games' wait for the first model reply rises from 9,000 s / 3,000 s to 21,600 s (6 hours), rerun only, so the longer caps cannot outlast the games' wait.

The extra RESETs fall on level 1 of one game: k = floor((L - 12.5) / 7) for a server first healthy at minute L, so 0 for a normal start and 7 for a 63-minute load; at most 0.087 points [computed, E154]. E154's first save, a real commit, waited 33 minutes for its server, so slow loads were live; the smoke passed [measured].

Its draws. E154 (kernel version 189, script 362115816) drew 31.00, 28.05, 27.07 and 29.43 (refs 57631952, 57701219, 57737027, 57765117; mean 28.89, SD 1.71 over four draws) and passes its registered board rule (first draw

> = 20; mean over >= 2 draws >= 26.11) [measured, E154, read 2026-10-30]. Against E098's six draws the difference is +0.78 (Welch t 0.69, p 0.51; with E098's SD taken as known, z 0.66, p 0.51; as a t on 5 df, 90% interval -1.60 to +3.16); four draws against six detect about 3.5 [computed, E138]. Its own SD shows no sign that the version is noisier or quieter than E098 (variance ratio 0.87, F(3,5) p about 0.97; descriptive) [computed]. Draw 1's +2.89 over E098 did not repeat. These draws show that the version scores normally, not that the keep-alive works: reruns return no log, so whether a slow load occurred in any rerun, and whether the keep-alive acted, is unknown [read, E154]. We claim neither a gain nor protection.

6.6 The submitted notebook

Table 3. Components of the pick-1 notebook (`{{PICK1_VERSION}}`).

Component	Origin	Evidence
Duck harness loop, Franzen's prompts, perception, guards, scheduler	inherited	E001
vLLM e975732 + overlay, Flash-Next NVFP4/FP8, MTP with 3 draft tokens, 32k draft vocabulary	inherited (the draft vocabulary from Lord Han Solo's source, attached by us since E051)	E040, E051
384 of 512 experts per layer	ours	E060, E073
constant-cost state hand-off	ours	E077, E081
K8V4 KV cache; window 160,768, reply cap 32,768, 19 games	ours	E083, E091, E098
retained functions kept on timeout	ours	E097, E098
keep-alive, start cap, launcher deadline, first-request grace (if E154 is pick 1)	ours	E154

Component	Origin	Evidence
fallbacks: fp8 KV at 15 games, then the unpruned server at 18 games	ours	E098

7. What one leaderboard draw can show

7.1 ARC-AGI-3

Six draws of E098 (two saved versions with an identical competition path; scripts 357974936 and 358364499) gave 27.27, 30.07, 28.24, 28.37, 29.68 and 25.01: mean 28.11, SD 1.83 (90% interval 1.23-3.82), standard error of the mean 0.75 [measured, computed; E138]. Before the sixth draw the SD was 1.14 (90% 0.74-2.70); the rise rests on the 25.01 draw, which is not a significant outlier (Grubbs G 1.69 < 1.89) [computed]. With the two draws of facts v1, the pooled within-version SD of our vLLM stack is 1.73 (90% 1.20-3.32) [computed]; the older SGLang notebook is noisier (28.56, 24.06, 21.87; SD 3.41) [measured]. E098's draws show no date trend (-0.25 points per day, p 0.41) [computed].

A single draw is therefore uncertain by about +-3.0 points at 90% (1.645 x 1.83) [computed]. For planning, at SD 1.73, one-sided 0.05, power 0.90 [computed, E138]: a true +3 needs 6 draws per version, +2 needs 13 and +1 needs 52. One new draw against E098's six detects 5.8 and two detect 4.4 at E098's own SD 1.83 (E138); at the pooled 1.73 they detect 5.5 and 4.1 [computed]. Extra draws cannot shrink the private half's game-set term [read: E138].

7.2 ARC-AGI-2

On the ARC-AGI-2 board (static grid puzzles, four L4 GPUs, twelve hours) the unchanged public 4B notebook (our v4) drew 30.97, 29.72, 31.81, 26.39, 28.06 and 29.31 (mean 29.38, SD 1.96); v5, which spends leftover time on extra views for low-confidence tasks (E132), drew 26.81, 30.97, 29.31, 28.47 and 28.89 (mean 28.89, SD 1.50) [measured, E138 update 11-01]. Pooled SD 1.77 (90% 1.29-2.91); v4 minus v5 +0.49, SE 1.07, p 0.66: not separable; a true 1.5-point gap would need 12 draws per version [computed, `analysis/leaderboard_noise_agi2.py` section 7]. v4's SD went from 1.05 to 2.19 with its fourth and fifth draws [computed]: three draws said little.

7.3 Decision rules

E138 registered a sequential rule: a candidate is adoptable after at least 3 draws and a mean gain of at least max(2.0, 2 standard errors); at the measured SD a true +3 passes 81% of the time and a true 0 about 3% [computed]. The final pick is the highest shrunk mean, never the best single draw [read, E138]. E154's board rule, applied to a four-draw mean, passes a version equal to E098 with probability 0.95, one 2 points worse with 0.50 and one 4 points worse with 0.04 [computed, E154]: a disaster filter that cannot detect a gain, which is why E154 claims none.

7.4 A lesson: the borrowed SD

Before our own repeats we used SD 3.9 from copies of the public notebook plus one SGLang repeat [recorded: ledger header]. Under it the facts-v1 version's -3.49 (two draws) looked uninformative (p 0.14); under our stack's SD the current -2.87 is suggestive (one-sided p 0.045, post hoc; exact Mann-Whitney p 0.071) [computed, E138]. A borrowed noise floor can label a suggestive difference as noise.

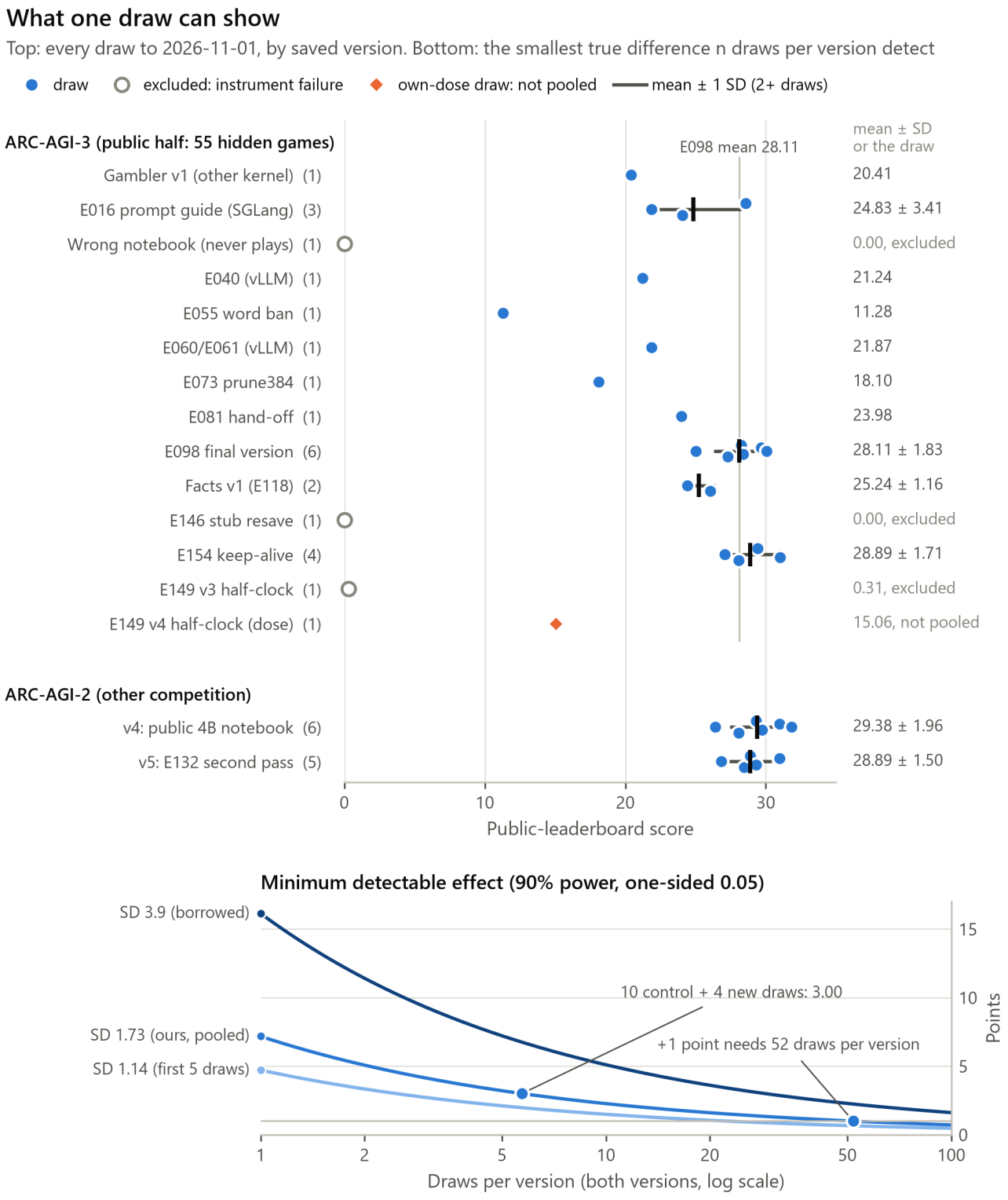


Figure 2. What one draw can show. Every leaderboard draw to 2026-11-01, by saved version, both competitions (E138, E154, E149) [measured]; mean +- 1 SD. E154 has four draws; ARC-AGI-2 v4 and v5 have six and five. Hollow: instrument failures, excluded (the 10-05 0.00, E146's 0.00, E149's 0.31). Diamond: E149 v4's half-clock 15.06, never pooled. Bottom: the minimum detectable effect against draws per version at SD 1.14, 1.73 and 3.9 [computed, E138].

8. What bounds tokens per game: memory and throughput

8.1 Serving points

Table 4. Serving points of the versions that drew on the hidden games.

Version	Games at once	Window / reply cap	Generated tok/s	Tokens per game in 9 h [computed]	Hidden-set draws
E073 (pruned, old code)	24	94,208 / 12,288	987.0 at 18 busy (E073)	not computed	18.10 (n 1)
E081 (pruned + hand-off)	24	94,208 / 12,288	1,112 (60-min screen)	~309k	23.98 (n 1)
E098 (+ K8V4, depth, timeout fix)	19	160,768 / 32,768	530.9-574.7 (E137 bench, 19 in flight); ~540 in play (E110)	~148-160k	28.11 (n 6, SD 1.83)
E154 (E098 + keep-alive)	19	160,768 / 32,768	as E098	as E098	28.89 (n 4, SD 1.71; Section 6.5)

Tokens per game = tokens per second x 8.5 hours of decoding / 110 games [computed]; the 8.5 hours is a guess that cancels in the ratio of two versions [guess]. E098's bench speeds come from its exact server in E137 (two windows 8.2% apart) [measured]; the in-play figure divides E110's 3.88M tokens by about 120 minutes [computed].

E098 decodes at about x0.49 of E081's speed (bench x0.48-0.52, in play x0.49) [computed]. By E040's step model, 24 -> 19 streams alone costs about x0.89 [computed; model fitted at short contexts]. The rest of the x0.49 is per-token work that grows with context (E084: throughput fell above 24 streams with the pool 63-83% full) [measured; the attribution is an inference]. Depth was paid for mainly in attention per token; by "memory" we mean the KV pool that bounds games x context. The KV format is not the cause: K8V4 left decode speed unchanged at the same contexts (E083) [measured].

8.2 Half the tokens, no loss

E098 received about half of E081's tokens per game (ratio 0.48-0.52) [computed] and scored 4.1 points more: 28.11 over six draws against 23.98 over one [measured]. Two readings of the public curve bracket what tokens alone predict (Figure 3) [computed, fig1 data]:

- Constant elasticity: stretching 0.59 from ~309k down to ~154k predicts 15.9 (12.9-19.0 across its 90% interval), an extrapolation over a range that is half unmeasured.
- Saturation: if returns stopped at 180k, E098 would still lose the 180k-to-~150k step and land at 21.5-21.9.

A confounded hidden-set reading supports saturation above about 180k: a vLLM version at about 250k tokens per game drew 21.24 while the SGLang notebook at about 180k averaged 24.83, an elasticity of -0.48 (SE 0.63) [computed, E072 round table 7; different servers and harnesses], and the public curve's last step is unstable (Table 2). Saturation explains part of the gap, not all: 28.11 is above both predictions, though E081's single draw is itself uncertain by about +-3.0 points (90%) [computed]. The supported finding is narrow: **halving tokens per game, a side effect of deeper turns, did not cost score; what bought the +4.1 is not isolated.** The halving was not planned: E098's registration assumed x0.9 of E081's tokens per game (Section 8.4) [read, E098]. Under the horizon argument this is the mirror image of the ban: E098 allowed more thinking per turn (a 32k reply cap, a longer window) at fewer tokens per game, and was not punished at deployment length [inference]; it was never screened short, so it is not a test of P1.

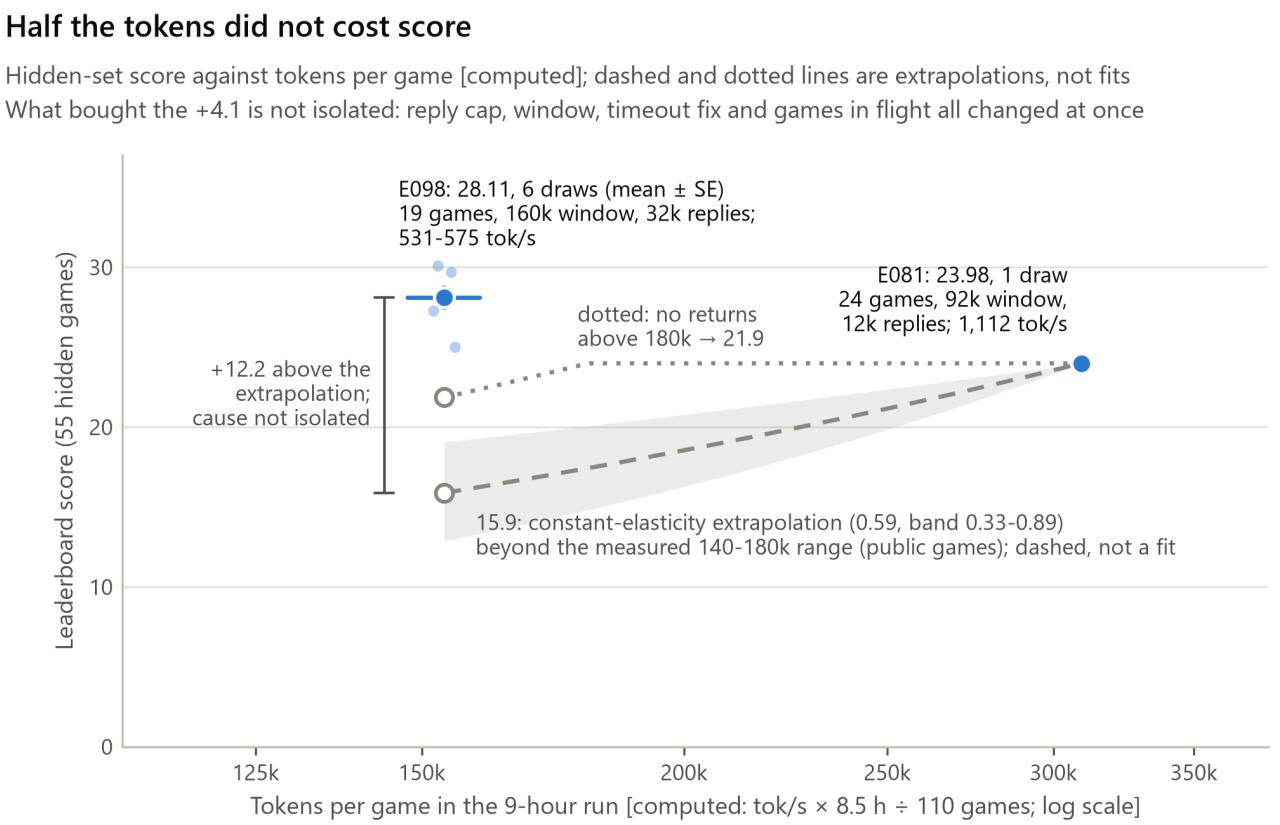


Figure 3. Half the tokens did not cost score. Hidden-set score, E081 one draw, E098 six +- SE [measured]. Tokens per game [computed: tok/s x 8.5 h / 110]. Dashed: elasticity 0.59 (E072) extrapolated, not fitted; dotted: zero returns above 180k. Confounded: reply cap, window, timeout fix, games in flight (and the KV format, Section 8.3).

8.3 Confounds

The E081-to-E098 step changed five things at once [read, E098]:

1. **Reply cap 12,288 to 32,768 tokens.** At 12k, 4.5-5.1% of replies in the public 60-minute controls had been cut off (E077) [measured].
2. **Window 94,208 to 160,768 tokens** (81,408 kept after a trim instead of 40,704). At 92k, E040's forgetting audit counted 4.7% re-derivation after trims [measured].
3. **The timeout fix**, a bug fix with no relation to memory (6 of 6 timeouts wiped functions in E081) [measured].
4. **Games at once 24 to 19**, which changes how the scheduler shares time.
5. **The K8V4 cache** itself (+0.0016 nats per token) [measured].

The E081 draw (10-12) and the E098 draws (10-13 to 10-21) also fall on different days [measured], though E098's draws show no date trend [computed]. Any of the five could carry the +4.1.

8.4 What was registered

E091 registered "on the hidden games, depth per game is worth more than the throughput it costs", with the cost then estimated at x0.8: SUPPORTED if the deeper mean is at least the shallow one + 3 over at least 2 draws each, CONTRADICTED if it is at most equal [read]. E098's registration predicted 22-27 (centre ~24.5) at an assumed x0.9 [read]. The twofold cost was found after the draws (skeptic v1, from E137 and E110). The +3 is met; the two-draw condition is not, since E081 never got a second draw.

8.5 Tests of the token leg

E149, half-clock (registered, void). With E098's five-draw mean (28.73) as reference, before the sixth draw was read, we registered a draw of E098 with half the time per game (266 of 532 minutes) and the model-visible clock doubled, so that only the token dose changes. Band for one draw: 13.5-24.8 (28.73 x 0.5^(0.89 to 0.33) +- 1.645 x 1.21; 11.9-25.6 at the current E098 record); below 5 void [read, E149]. Only a score above the band would have contradicted the public down-slope. Built on the E154 base, it drew 0.31 (ref 57669233): void by its own rule, nothing read [measured]. No code cause was found; one candidate is that game clocks start at release, before the model loads, so a very late load leaves only the half-clock version without play [inference from a CPU simulation of the rerun path].

E149 v4 (kernel version 191). Every game's 266 minutes count from the run's first model reply, with a run deadline at E098's own end (notebook minute 544): dose 266 / (544 - L), 0.500 for a server first answering at minute 12, 0.524 at minute 36, 1 after minute 278 [computed]. It passed its CPU tests (32/32) and, on 2026-10-30, every item of its registered smoke rule in a real commit (that commit's server took 10 minutes); draw 1 (ref 57806285) was submitted the same day [measured, E149 'Result, v4 smoke and draw 1']. Band and reading as registered; never pooled with any other version.

Result [measured, Kaggle API read 2026-11-01; computed, `analysis/leaderboard_noise.py` section 13; E149 "Result, v4 board draw 1"]: ref 57806285 (script version 363866748) scored **15.06**, COMPLETE. Not void (y >= 5). It lies inside the primary band 13.5-24.8 (1.55 above its lower edge) and inside the band restated at the E098 record, 11.9-25.6 (3.14 above its lower edge). By the registered words, the public down-slope transfers to the hidden 9-hour run (down side). Elasticity: e-hat = ln(15.06 / 28.73) / ln 0.5 = 0.93 +- 0.11 at the registered SD 1.14 (90% interval 0.75-1.12); against the E098 record 28.11 at its SD 1.83, 0.90 +- 0.18 (90% interval 0.61-1.20). The effective dose 266 / (544 - L) is 0.500 for a server first answering at minute 12 (e-hat 0.93) and 0.524 at minute 36 (1.00); L is not observable, and e-hat rises with it (its minimum, 0.90, is at L = 0). The draw sits at the steep end of the public range: 0.04 above its upper edge 0.89 and 0.34 above its point estimate 0.59 (about 1.7 combined standard errors) [computed, approximate]. My registered guess was y about 19 (80% interval 15.5-23); the draw is 0.44 below that interval and inside the band. With the token leg holding on the hidden games, E098's +4.1 at half the tokens means depth, with the changes that moved with it, paid for the halving [inference; not isolated]. Caveats: one draw; the SE uses E098's SD, which a half-dose version may not share, so it is a lower bound [guess]; whether the clock-origin patch acted, and the load time L, are not visible on the leaderboard; it is a down-dose and cannot price a speed-up (E148 (e)). Against v3's void 0.31 the difference is +14.75 between two single draws of two versions: consistent with the clock diagnosis, not a test of it; no cause is attributed.

E110 truncation test (E155, registered 10-28 before the data were downloaded). It truncates E110's run of the unchanged E098 version on 38 community games (Section 11) at fixed token budgets and measures completion S(B), the scoring rule's level weights with the move factor 1 (community games have no human counts). Estimate: e = ln(S(100k) / S(50k)) / ln 2, game-cluster bootstrap (10,000 resamples, seed 110), against the same metric on the two public 60-minute controls. If fewer than 30 games reach 100k, the upper budget drops to 90k or 80k, chosen from token counts alone. HOLDS if e >= 0.33 with its lower bound above 0; WEAKER if e < 0.33 with its upper bound below the public value; else inconclusive; void if no budget qualifies, the data are not obtained by 11-04, or E110's totals (150 levels, 3.88M tokens) are not reproduced within 1% [read, E155]. Prediction: e about 0.6 (80% interval 0.25-1.0) [guess]. It tests the token leg at deployed depth on a second game set, not what bought the +4.1. Result: `{{E110_TRUNCATION_RESULT}}`

8.6 Memory prices each speed lever

- **Pruning:** x0.991 per stream, 18 to 24 games (Section 6.1) [measured, computed].
- **A newer drafter (DFlash2)** needs 97 pool blocks per request for its own sliding-window cache on top of the model's 79-86, so 9 full-context games fit instead of 19 (the server's own allocator, whose MTP row

reproduces the logged pool; E137 T2a) [computed]. It never served on our stack (a config guard, then a block-alignment assertion; E137 v1, v2-lite) [measured], and its registered fit rule (fit 9 < 17) killed it before a third GPU run.

- **A fourth draft token:** x0.938 in one 300-second window per arm, while two identical MTP windows differed by 8.2%: no gain shown (E084, E137) [measured].
- **Width above 24 streams:** x0.927 at 32 streams (step 51 to 73 ms), falsifying our own throughput model (E084) [measured].
- **A down-dose cannot price a speed-up:** on a simulator matched to the measured slope, x1.15 tokens gains 0.0-0.9 points instead of 1.4-3.8 at constant elasticity (E148 (e)) [computed on a simulator that overshoots 60-minute returns].

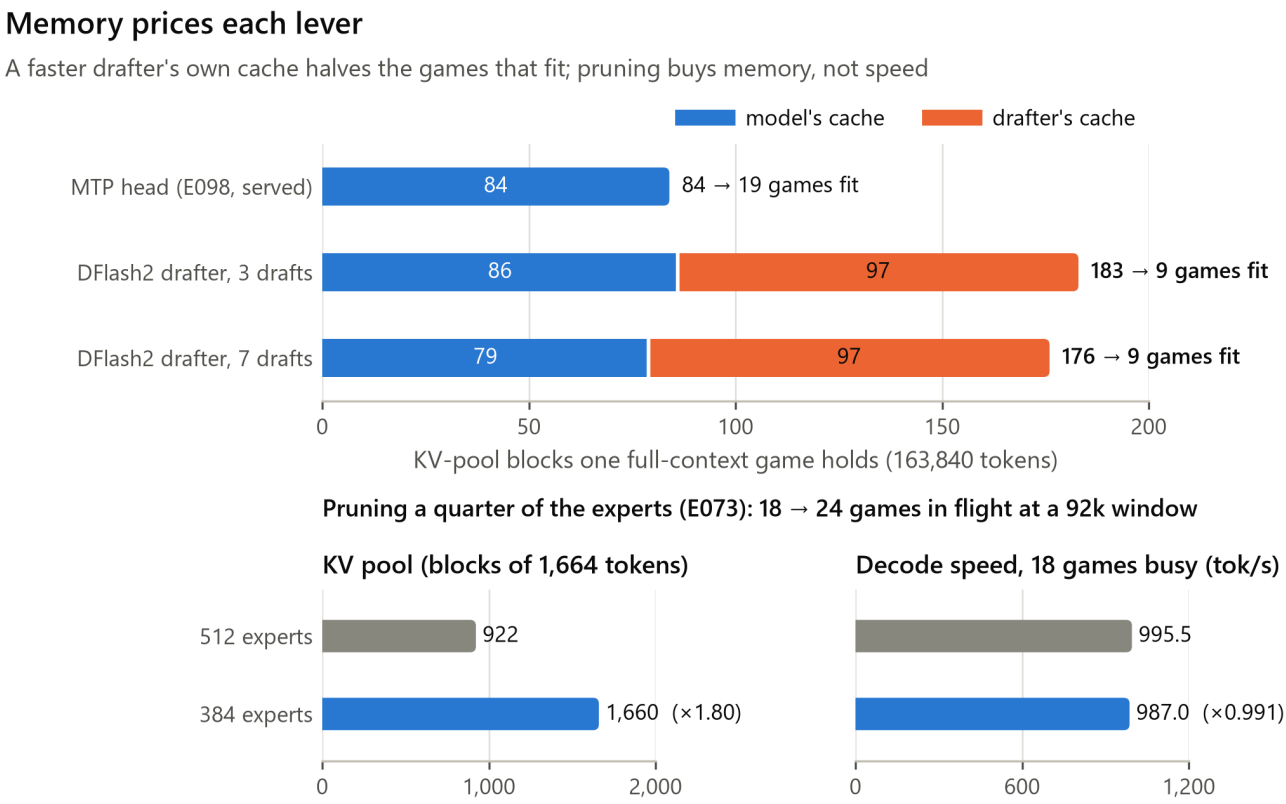


Figure 4. Memory prices each lever. Pool blocks per 163,840-token game and games that fit [computed with the server's allocator; E137]. Bottom: pruning (E073): pool [computed from measured GiB], decode speed [measured], 18 to 24 games [computed fit].

9. Where the agent fails

The model usually reads the goal; it fails by fixing a wrong rule from one observation and never running the deciding test. Four hand-read games of the 20-minute E040-era agent, not E098, show the pattern (E047, E048) [read in transcripts, not measured statistically; the pattern is an interpretation]. Under strict blind coding of 15 same-length stalled-vs-winning pairs, the stalled run held the winner's complete rule in 3 (90% 0.08-0.41) and had acted on it in none; lenient coding gives 12 of 15 (E072, T4 strict) [measured]. Most stalls lack the rule.

At 16 logged wrong-belief turns, writing the exact facts into the start of the model's thinking cut the wrong-belief rate from 0.58 to 0.23 but raised right decisions by only +0.03 (CI -0.13 to +0.17); a generic re-check sentence made them worse (-0.11) (E102) [measured]. Continuing the full agent from 13 such points, one hand-written deciding fact raised completion by +0.22 (95% CI +0.04 to +0.41; E112), while harness-generated facts

as deployed moved it +0.05 (CI including 0; E136) [measured]. Beliefs move; completion moves only when a person points. The first action sampled at a decision does not detectably predict completion (exact-context share 0.17, p 0.38; E115 T0) [measured].

10. Pre-registered negatives

Every row was registered with its kill rule before its first number. "Kill number" is the measured value that decided it.

Table 5. Ideas that did not survive their own rule.

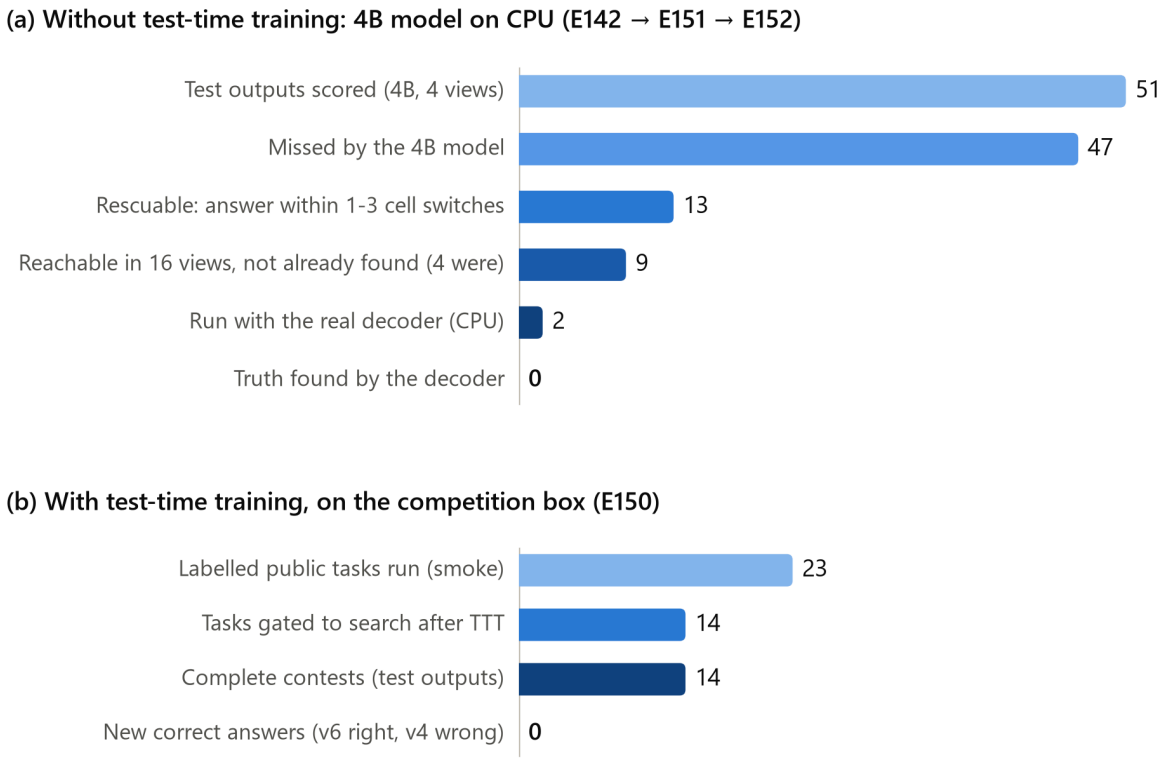
Idea	Entry	Kill number [measured unless noted]	Verdict
Field guide from game source code + stall critic (prompt)	E016	28.56 / 24.06 / 21.87, mean 24.83 vs 27.80 / 27.89 unchanged	no evidence of help
"Gambler" prompt	E012	20.41 vs 27.80 / 27.89	dropped
Per-turn hint lines ("switches")	E036	59 vs 64 levels (drop at <= control - 5); named control used 57% vs 60%	inert
Worked-case prefix from frontier play	E050	positive control 0/3 < 2/3; held-out 51 vs 51 levels	dropped
Facts text at decision points (fact line, own-voice seed)	E102	flip +0.05 (CI -0.09, +0.19), +0.03 (CI -0.13, +0.17)	not candidates
Harness-computed facts (events, surprise, map)	E097	73 vs 76 levels	inconclusive, not adopted
Generated facts as deployed	E136	+0.05, CI includes 0	KILL
Facts v1 on the board	E118	24.42, 26.06; -2.87 vs E098 (one-sided p 0.045) [computed]	out of the pick
Medium reasoning effort	E042	45 vs 53 levels	DROP
Self-check word ban	E055, E058	-20 levels at 60 min; 11.28 on the hidden games	DROP
More games at once (> 24 streams)	E084	x0.927 at 32 streams	DEAD
Fourth draft token	E084, E137	x0.938 (one window; windows vary 8.2%)	no gain shown
DFlash2 drafter	E137	never served (2 instrument kills); fit 9 < 17 [computed]	KILL before GPU
Action-level RL gate	E115 T0	exact-context share 0.17 (90% 0.00-0.54), p 0.38	RL line stopped
Cheap observables to pick the better opener	E131	no signal in 6 of 6 selectors	dead
Light fine-tuning (top 4 layers, own fewest-token wins)	E134	held-out +0.056 (90% -0.008, +0.119) vs bar +0.08	DEAD (a gain of the bar's size not excluded)
Half-clock dose draw	E149 v3	0.31, below the void threshold 5	VOID
Stub-commit saves (v1)	E146	0.00 rerun	BROKEN; real commits used
ARC-AGI-2: Flash-Next on 4x L4	E129	1/40 = 2.5% < 20%; 92 tok/s	KILL
ARC-AGI-2: Qwen3.8-27B reasoning	E135	pass@2 1/26 = 3.85% < 20%; needs 4.63 h > 3.83 h	KILL
ARC-AGI-2: looped models for TTT	E133	best looped arm +0.001 NLL (CI -0.014, +0.018) at 2.5x cost	KILL
ARC-AGI-2: icecuber DSL search	E141	3/120 < 6	KILL
ARC-AGI-2: >200k ARC-AGI-1 programs	E144	1/120, already solved	KILL

Idea	Entry	Kill number [measured unless noted]	Verdict
ARC-AGI-2: backward scoring of candidates	E145 v2	+0.038 (lower bound 0.000) vs +0.05	WEAK
ARC-AGI-2: 27B in a Python loop	E147, E147 v2	0/18 + 0/7 + 0/24 = 0/49 task-runs; pass@2 0.0 < 0.15	KILL
ARC-AGI-2: limited-discrepancy decoding (the "v6 ladder")	E142, E151, E152, E150	reachable 9/9 without TTT; found 0/2 (E152); with TTT 0/14 contests, NET 0 (E150)	KILL
ARC-AGI-2: continued fine-tuning of the 4B (Plan B)	E153	registered, never run: closed by decision on 10-27 (no cloud spend) [read: project decision log]	closed

The pattern across the ARC-AGI-3 rows: interventions on what the model is told (prompts, hints, cases, facts) did not change what it does; interventions on how much it thinks changed it in the wrong direction at length; interventions on memory and the harness's own overhead were the ones that held.

Reachable is not found (ARC-AGI-2)

Two separate task sets: the answer was within reach of a small decoder change, yet no search found it



Overlap: 7 of the 13 rescuable tasks of (a) are among the 14 gated tasks of (b). NET correct answers vs the 4B notebook: 0

Figure 5. Reachable is not found (ARC-AGI-2). (a) No test-time training: 51 test outputs, 47 missed, 13 within 1-3 cell switches (E142), 9 reachable in 16 views (E151), 2 run with the real decoder, 0 found (E152). (b) Test-time training on the box (E150): 23 tasks, 14 gated, 14 contests, 0 new correct answers. Different task sets [measured].

11. Beyond the public games

The principle without ARC words. When actions are scored and thinking is free on fixed hardware, a test shorter than deployment favours agents that think less, so a change to how much an agent thinks should be judged at the deployment horizon; what bounds the thinking is the memory each episode holds and the price of

each token at its context length. Settings with the same shape include laboratory or robot trials run by an agent, probing a system with a request budget, or diagnosis where each test is costly, and any setting where a cheap pilot stands in for a long deployment [guess: not tested by us].

A second game set. E110 ran the unchanged E098 version for about 60 minutes on each of 38 community-made games it was never tuned on (25 DreamTeam, 13 ARC-Interactive; vetted by fixed rules before any was looked at). It finished 150 levels (3.95 per game) with 3.88M tokens (102k per game; 38.6 levels per million) [measured]. The E081-era version on the public 18 games (E097 control, another session) gave 4.22 levels per game and 22.9 per million [measured]: the level rate transfers (x0.94), and community levels are x1.69 cheaper per token [computed]. This crosses game sets and versions, so it shows the agent transfers; the E110 truncation test (E155, Section 8.5) tests the token leg at deployed depth.

No public-set score for the final stack. The ledger holds no run of the E098 or E154 version on the public games at deployment length. Their only public-game plays are the commit smoke checks, 4 public games x 5 minutes (E154's: 10 levels; E149 v3's, the same stack with the half clock: 8) [measured, E149 diagnosis], which test wiring, not skill. GPU quota, the project's scarcest resource, went to screens of candidate changes and to E110; the final stack itself was judged on hidden-set draws, which use no quota (E098, E154) [read]. The nearest numbers are the E081 version on the public 18 games at 60 minutes (4.22 levels per game, E097 control) and the E098 version on 38 community games (3.95, E110).

A second benchmark. The same 125B model made 92 tokens per second on ARC-AGI-2's four L4 GPUs, about 33k tokens per task, under the more than 49k it needs for one finished answer (E129) [measured, computed]; on the RTX PRO 6000 our versions made 531-1,112 (E098, E081) [measured]. The box picks the model. On ARC-AGI-2 we submit the unchanged public 4B notebook (v4) or its second-pass variant (v5); every search we added found no new answers (Table 5, Figure 5).

12. Related work

ARC-AGI-3 harnesses and agents. Tufa Labs' Duck harness (Cottaar; Kaggle discussion 717133) introduced the code-tool loop. Franzen's writeup credits token throughput, since his scores were still rising at the end of runs, and larger context [read: vendor/franzen-m2/WRITEUP.md]. Lord Han Solo runs his own harness on exactly our server stack [read, E087]. Liew (arXiv 2605.25931) argues that the 25 public games can be beaten by non-intelligent strategies, so only the hidden set tests exploration, and frames ARC-AGI-3 as a speed-depth trade-off between action efficiency and information gain (a three-phase agent on Qwen2.5-0.5B, RHAE 0.2116 on the public games) [read: abstract, 10-30]; the trade-off is over actions, ours over thinking tokens. Brown's blog post "how I accidentally got the top score on ARC-AGI-3 with 5.5x fewer tokens" (2026-07-29, https://blog.ryanbbrown.com/p/how-i-accidentally-got-the-top-score) reports 99.86% mean RHAE on the 25 public games with a frontier API model (GPT 5.6 "sol" at maximum effort) using about 5.5 times fewer tokens than an earlier harness at a similar number of actions (7,703 vs 8,347), so cost was set by tokens per action [read 10-30]. Its effort comparison moved effort and harness changes together, and the post reports no ablation of effort alone [read]. Both work on the public games; ours is a small offline model judged on hidden games, and we measure the returns rather than tune for them: the elasticity, the noise floor, the horizon dependence.

Serving economics. PagedAttention (Kwon et al., 2023) showed that KV memory caps how many requests batch together [read]; that is standard. We add the agent-side consequence: memory converts into score through games x context, at a per-token price for depth, with a measured exchange on one task.

Thinking length, budgets and horizons. Section 5.7 positions the reversal against NoWait, CGRS, s1, TTI and the budget-dependent rankings of de Souza and Panisson (arXiv 2608.12150); arXiv 2506.04210 and 2506.23840 are also single-answer [read]. Short-horizon bias was named in meta-optimisation (Wu et al., ICLR

2018, arXiv 1803.02021) [read: abstract, 10-30]; we see it in agent evaluation. The knowing-doing gap is documented (Schmied et al., arXiv 2504.16078) [recorded], though our strict count finds most stalls lack the rule (Section 9).

13. Limitations

- One model (Qwen 3.8 Flash Next) and one GPU type. The elasticity is measured on 18 public games at a shallower depth.
- The horizon argument was stated after the reversal it explains (Section 4.4); the analogy to short-horizon bias is ours, and the per-token attribution of depth's cost (Section 8.1) is an inference.
- The horizon reversal rests on one change (the ban), one 60-minute session and single hidden-set draws of non-identical versions. The other thinking cut we tried, medium reasoning effort (E042), lost even at 20 minutes with no early lead (Section 5.6), so a short test does not always favour a cut; the reversal is not shown to be general.
- The only registered test of the principle itself, E157 (Section 4.4, P5), is void by its mechanism gate (the post-trained model kept 0.93x our reasoning tokens); a valid single draw would have been underpowered below about 6 points and could detect only a collapse. E149 v4's pass is one draw of a down-dose and tests the token leg, not the principle.
- The hidden-set support for C3 is a single comparison: E098 with six draws against E081 with one; the tokens-per-game axis is computed from bench and screen speeds, not measured in the hidden runs; reply cap, window, timeout fix, games at once and the KV format changed together (Section 8.3).
- Leaderboard reruns return no logs: whether a slow load happened, or the keep-alive acted, is never observed.
- There is no public-set score for the final stack (Section 11).
- Our final score is level with the public baseline we started from and well below the top five (fifth place 51.51 and first place 75.56 on the public leaderboard, read 2026-10-30 [measured]).
- The Flash-Next weights carry the Qwen Community License; whether they are prize-eligible is an open question that affects most teams [recorded].

14. Reproducibility

Versions. ARC-AGI-3 pick 1: `{{PICK1_VERSION}}`. Public notebook: https://www.kaggle.com/code/rusheelsharma/arc-agi-3-short-tests-pick-the-wrong-agent-e154 version 1, submission 57877052: its 25 code cells are hash-identical to version 189's, plus one licence header cell; its smoke commit passed on 2026-11-01 after a 15-minute load [measured, ledger E138 block of 11-01]. Candidates: E154 = kernel rusheelsharma/arc-agi-3-e109 version 189, script 362115816, notebook `ARC-AGI3/arc3-e154-keepalive.ipynb` (sha256 6cf982cd...), built by `analysis/build_e154_nb.py`, tests `analysis/test_e154_nb.py` 66/66; E098 = scripts 358364499 and 357974936 (competition path identical by the build tests), notebook sha256 89807ca1..., built by `analysis/build_e098_nb.py`, tests 92/92 [read]. Never submit or pool with these: version 187 (stub, 0.00), version 190 (E149 v3, 0.31), version 191 (E149 v4, 15.06, a separate test) and E157's version. Inputs: the competition data, dfranzen/taaf-kaggle-source-bundle-copy, lordhansolo/vllm-main-e975732-arc3, lordhansolo/qwen3-8-flash-next-mixed-nvfp4-fp8, lordhansolo/taaf-kaggle-source, rusheelsharma/arc3-probe-set-e058 and rusheelsharma/arc3-e061-tools (E091) [read]. ARC-AGI-2: kernel rusheelsharma/failed-in-aimo, v4 = script 359703883 (a byte-identical copy of koushikrudra's public notebook) and v5 = script 359728161 (E132) [read, E138].

Settings. Sampling temperature 0.7, top-p 0.95, top-k 20 (E102, "sampling as logged") [read]; the inherited harness's sampler seed defaults to -1, i.e. unseeded (`LOCAL_ANALYZER_SEED`, vendor/franzen-m2), and the E098 and E154 notebooks do not set it (its only occurrence there is that default) [read], so draws of one version differ at least by sampling. 19 games at once, window 160,768, reply cap 32,768, server max-model-len 163,840, K8V4 with 2,128-token blocks, 3 MTP draft tokens, 384 experts per layer (E098) [read].

Analysis. Leaderboard statistics: `analysis/leaderboard_noise.py`, `analysis/leaderboard_noise_agi2.py`. Public returns: `analysis/t3_recheck.py` (10,000 game-cluster resamples). Ban curves: `analysis/per_game_totals.py`, `analysis/k_killtests.py`. Figures: `analysis/make_paper_figures.py` (CPU, about 25 s); cover: `reports/paper_figures/make_cover_v6.py`. Bootstrap seeds: 134 (E134), 110 (E155). Crash testing: `localrig/` with a stand-in model.

Compute. Kaggle's free tier only: one RTX PRO 6000 for ARC-AGI-3 development and four L4 GPUs for ARC-AGI-2; under 150 Kaggle quota-hours in total, at most five weekly quotas of 30 (RTX time burns about 1.75 quota-hours per hour, the four L4 about 2) [estimate; the ledger records per-run costs, not a total]. Leaderboard draws use no quota.

Release. Code, ledger and this paper: https://github.com/Rusheel86/arc-agi3-short-tests, under MIT-0 for our code; inherited files keep their licences (Table 1). Unlicensed material (BARC seeds; NVARC code and data) is not redistributed.

Appendix A. Claim index

Claim	Entries	Strength
Cutting thinking leads early, loses late (observed, not predicted)	E055, E058, E072	one 60-min session; hidden set 1 draw each
A short test is biased toward less thinking (the horizon argument)	E058 mechanism; Wu et al. 2018	argued, stated post hoc; record in Section 4.4
Public return to tokens, 0.59 (0.33-0.89) between 140k and 180k	E072	two 60-min runs, 18 public games
Half the tokens per game for depth did not cost score	E098, E081, E137, E110	6 draws vs 1; axis computed; 5 confounds
Depth's throughput price is mostly attention per token (x0.89 vs x0.49)	E040, E084, E137, E110	inference from measured rates
A single draw is uncertain by about +-3 points at 90%; a true +3 needs about six draws per version	E138	6 draws of one stack; SD interval x3 wide
AGI-2 versions not separable	E138	6 vs 5 draws, p 0.66
Hand-off restores occupancy	E077, E081	one 60-min screen, replay byte-identity
K8V4 is quality-safe and memory-only	E083	probe on 60 requests, one bench
Keep-alive version scores normally (the keep-alive's own effect not shown)	E154	4 draws, mean 28.89; reruns return no log
The public down-slope transfers to the hidden run (registered, in band)	E149 v4	one draw, 15.06; e-hat 0.93 +- 0.11
A model that thinks less loses at full length (registered prospective test)	E157	VOID by its mechanism gate (0.93x reasoning tokens)
Most stalls lack the rule	E072	15 pairs, strict blind coding
Prompts and generated facts inert	E016, E012, E036, E050, E102, E136	registered kills
Search on ARC-AGI-2 found no new answers	E141-E152	registered kills